Machine Learning Based Algorithm Selection for Sorting Numeric Arrays

Official MSc thesis defense

93.1% runtime gap closed	**76.1%** test accuracy	**89.6%** zero regret

Ahmed Salah Abdalhi Mohammed

Supervisor: Assoc. Prof. Dr. Emre Özbilge - Cyprus International University - June 2026

The problem is choosing the right algorithm for ea

The thesis question is not how to sort, but which sorter to trust for this ar

Same complexity does not mean same runtime

O(n log n)

same high-level complexity class

introsort

heapsort

timsort

The practical winner depends on input structure, constants, memory behavior

This thesis selects before sorting begins

Array

not sorted yet

16 features

cheap structural summary

one pick

then execute

The selector pays a small feature cost to avoid a larger runtime m

The aim is a practical selector, not a perfect oracl

93.1%
gap closed

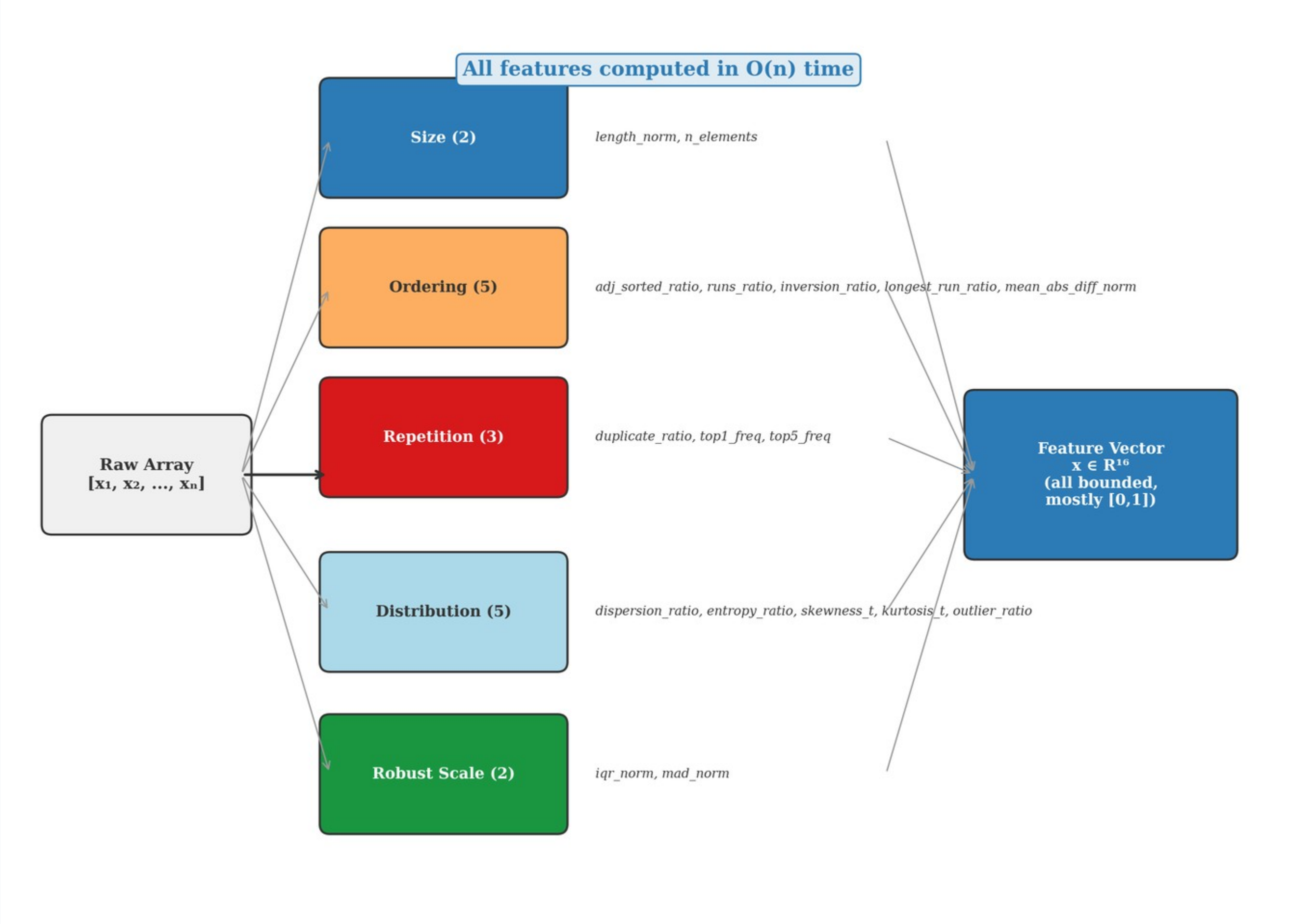
76.1%
accuracy

89.6%
zero regret

A wrong label is only damaging when it creates meaningful runtime regre

That is why the defense leads with runtime value, then explains accuracy and limits.

The array is converted into structural signals



The model does not see t

It sees compact structure:

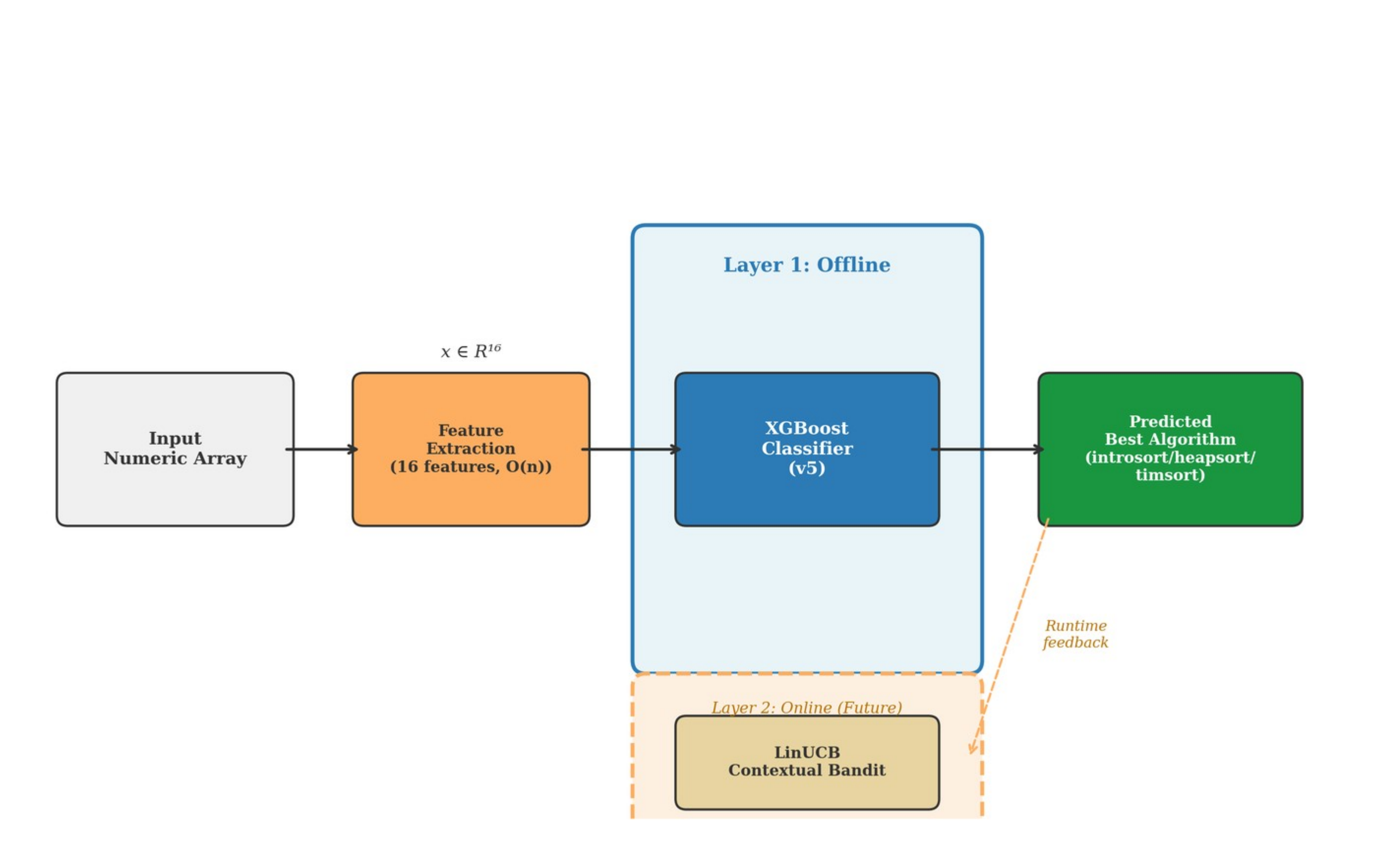
size

ordering

repetition

distribution

The system makes one decision before execution



features

XGBoost

one of three algorithms

No multiple sorting runs are used at de

The algorithms are different enough to make selec

introsort

hybrid quicksort path

strong average behavior

heapsort

stable worst-case bound

often the SBS baseline

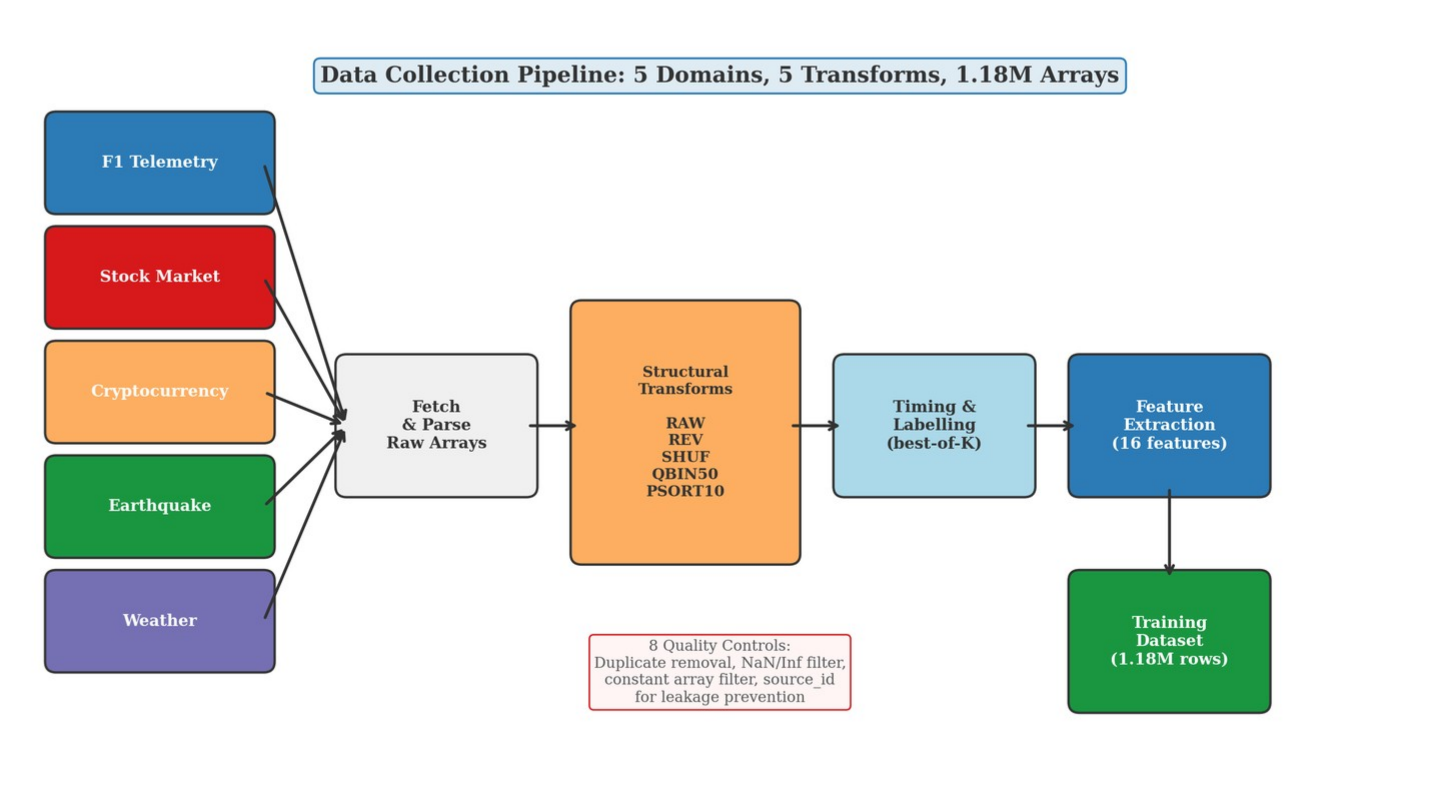
timsort

uses existing runs

structurally visible

The portfolio is small enough to defend and different enough to learn.

Real arrays give the selector the structures synth



1.18M

arrays before filtering

5 domains

F1 telemetry

stock, crypto

earthquake, weather

Source: metrics_summary.json M5_dataset_statistics

The labels come from measured runtime, not opin

Generate array
real domain plus transforms

→

Benchmark all three
same array, measured time

→

Label fastest
with noise filtering

Transforms used: **RAW** **REV** **SHUF** **QBIN50** **PSORT10**

The model learns from observed winners, not from a manual rule about al

The features describe structure, not raw values

Size

length_norm

Repetition

duplicate_ratio, top frequency ratios

Ordering

runs, inversions, adjacent sortedness

Distribution

entropy, skewness, dispersion, outliers

16 features summarize what the sorting algorithms can exploit.

The main model is the general v5 selector

XGBoost v5

16 structural features

3 algorithm labels

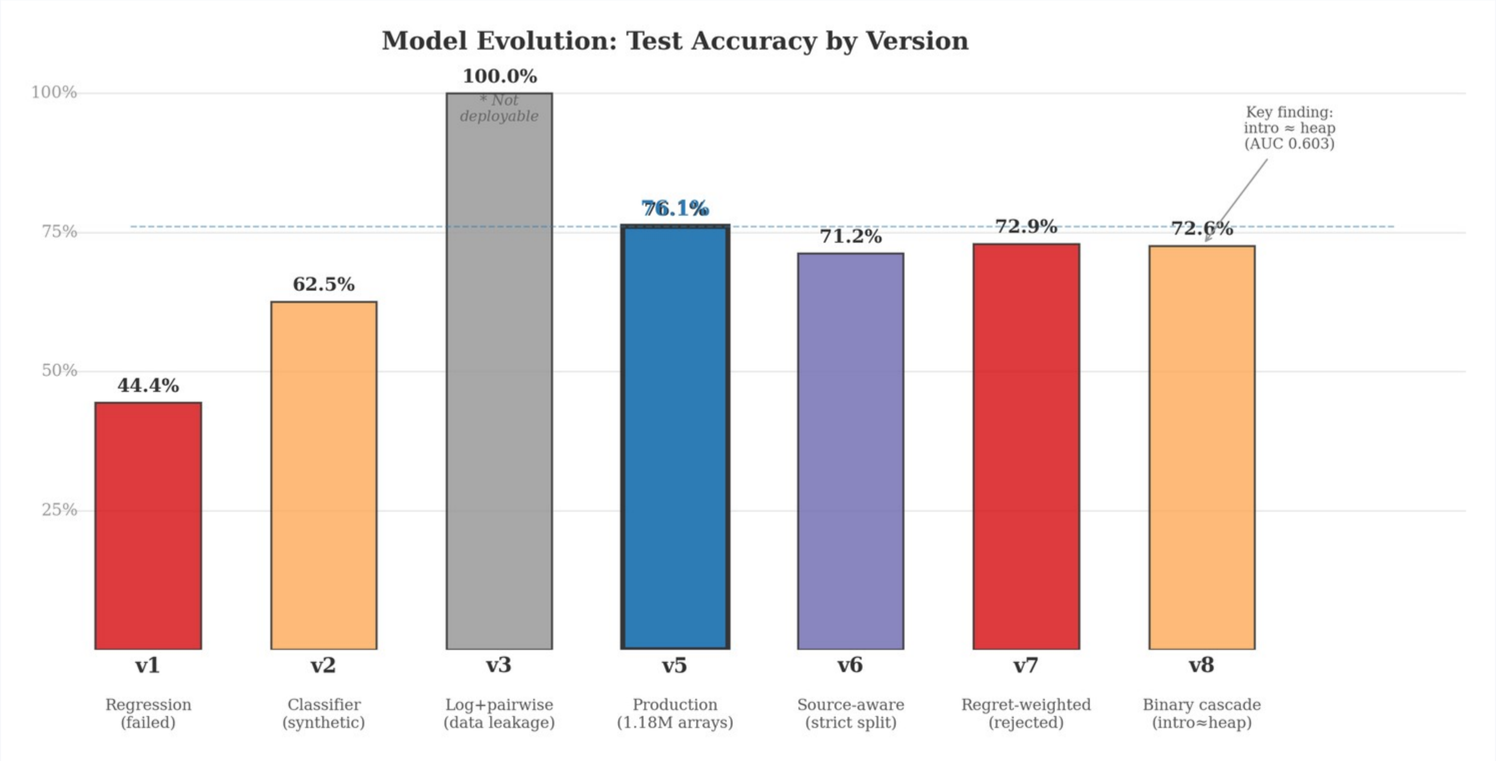
70 / 15 / 15 split

balanced training strategy

Separate from F1 routing

v5 is the general thesis model. F1 channel models are a specialized experiment.

Each failed path clarified the final design



What changed

regression was weak

timing features leaked

regret weighting hurt

cascade exposed boundary

The journey matters because it explains why v5 is the defensible center of the thesis.

The model is imperfect, but most mistakes are ch

93.1%

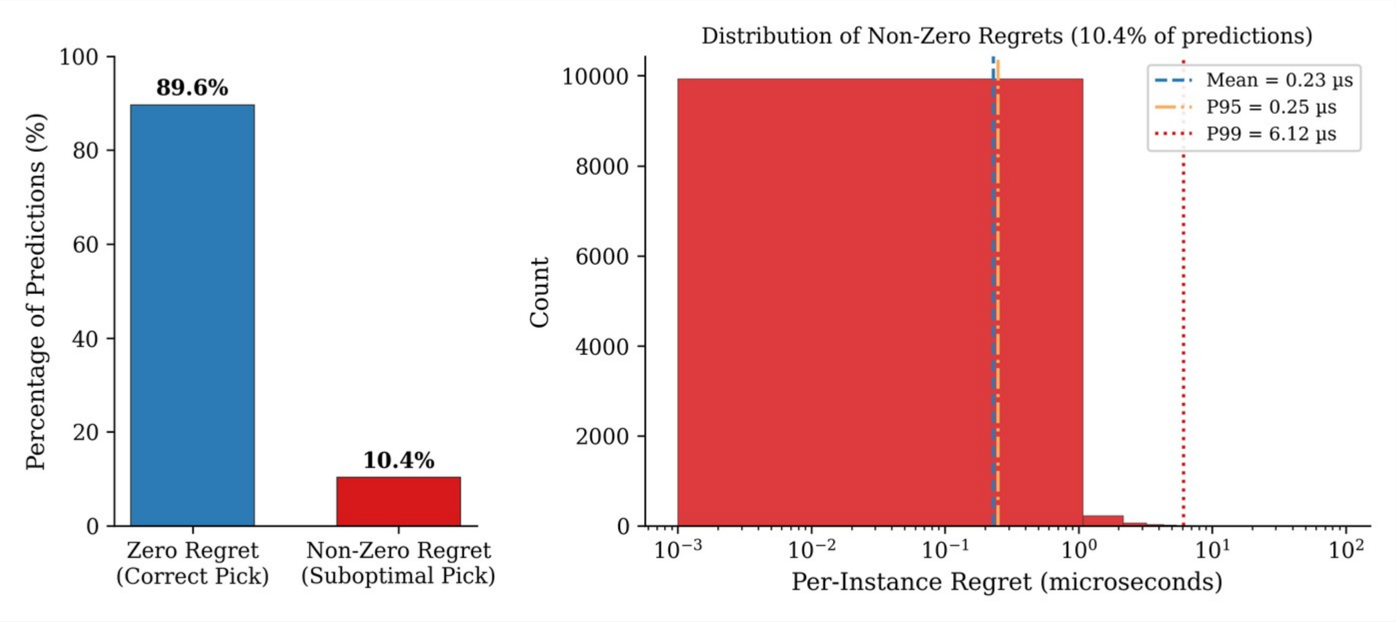
of the SBS-to-VBS runtime gap closed

runtime value first

Supported by 76.1% test accuracy and 89.6% zero-regret selections.

Source: metrics_summary.json M1_v5_evaluation and M2_v5_regret

Accuracy alone does not explain the value

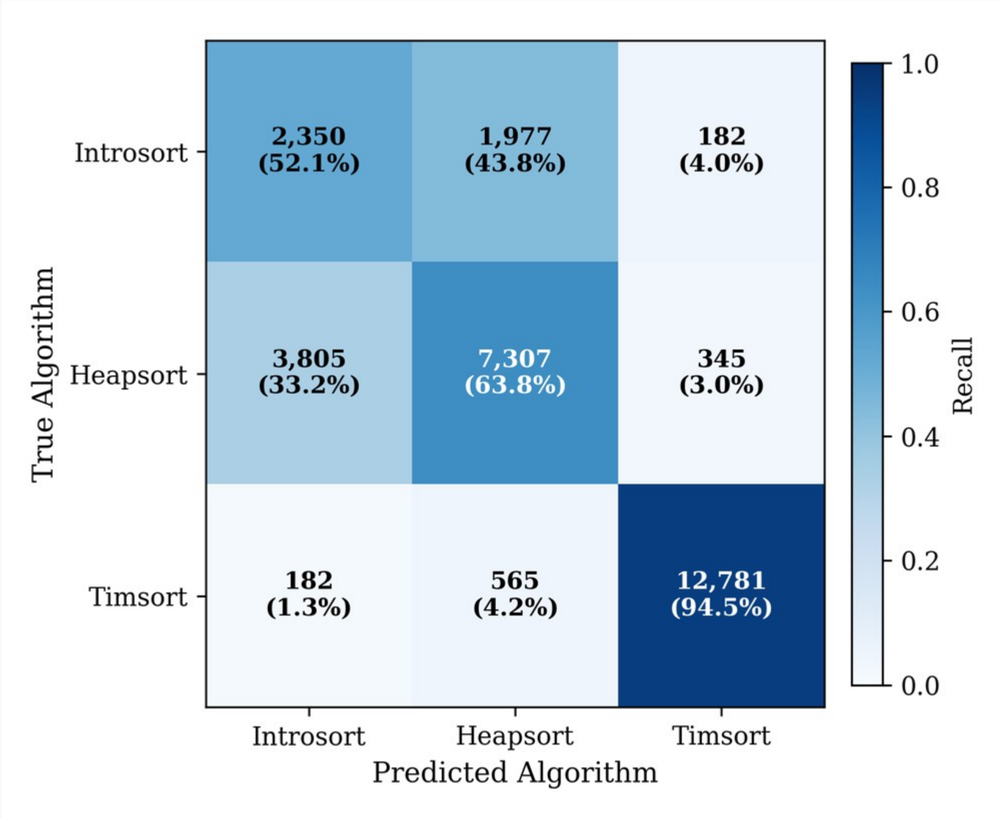


$$Regret = \frac{T_{model} - T_{VBS}}{T_{VBS}}$$

$$Gap\ Closed = \frac{T_{SBS} - T_{model}}{T_{SBS} - T_{VBS}}$$

A selection can be wrong as a label and still almost right as a runtime deci

Timsort is structurally visible; introsort vs heapso

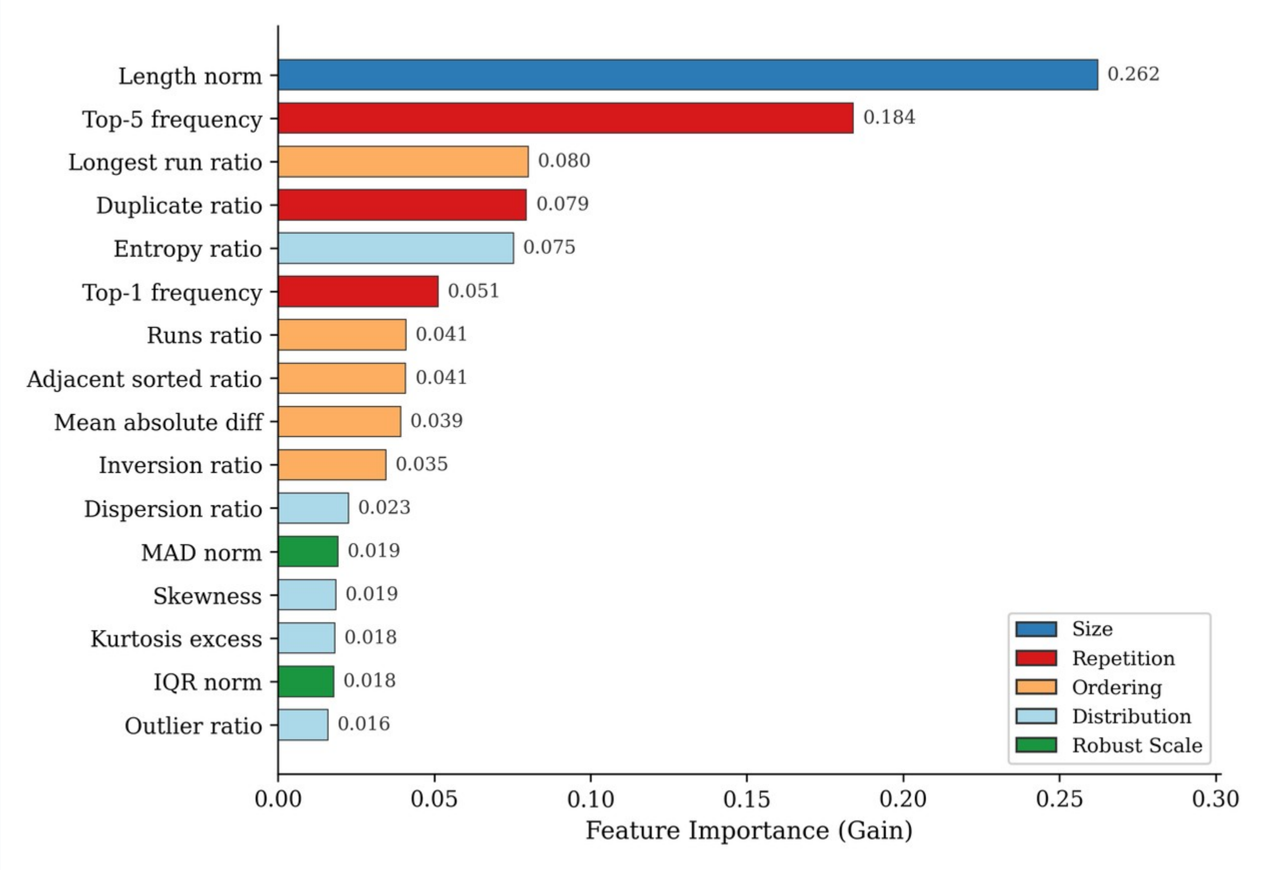


94.5%

timsort recall

The weak boundary is between introsort and heapsort, where structural features are close.

The model mostly uses features that make sorting



Top signals

length

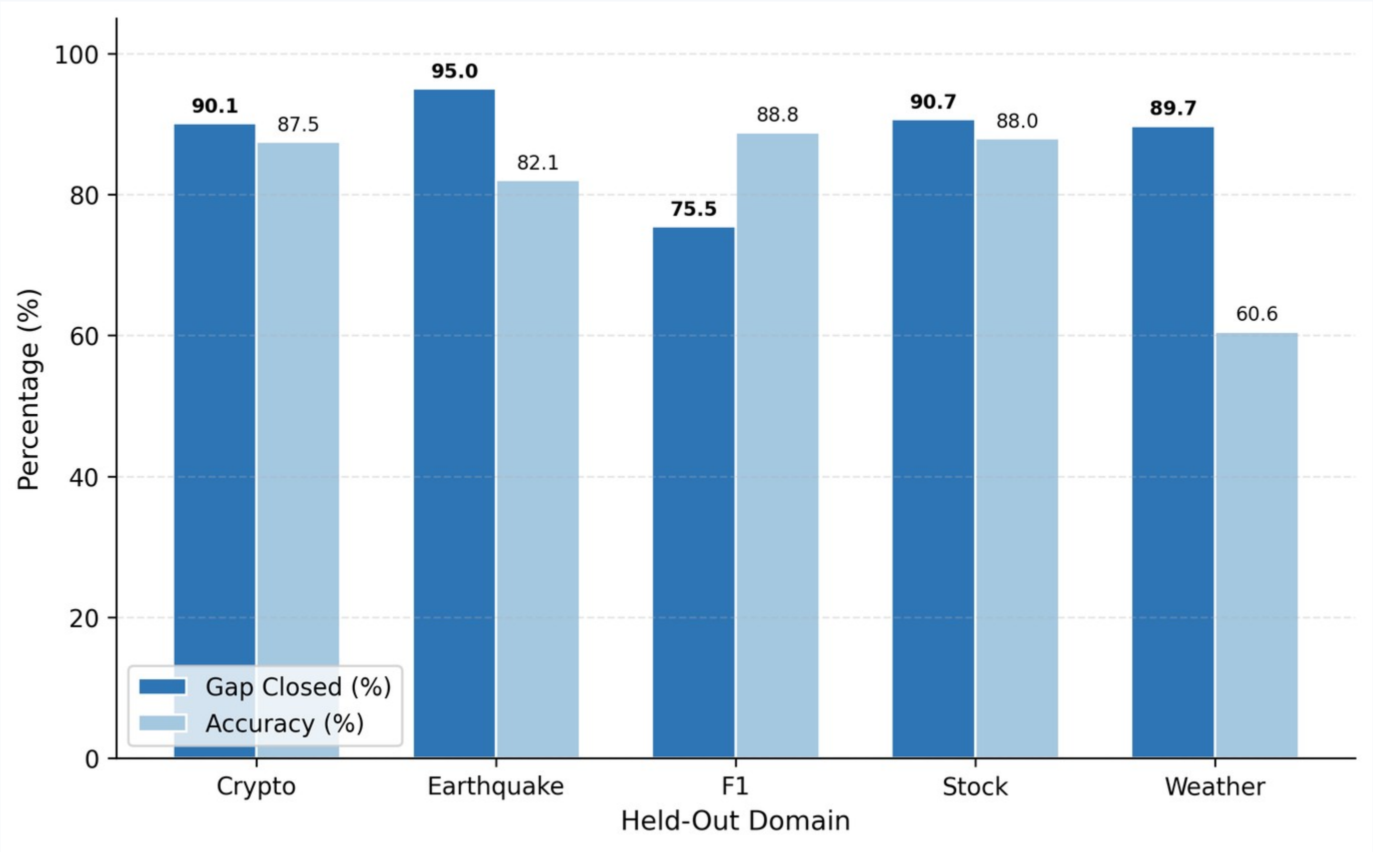
top frequency

longest run

duplicate ratio

This is interpretation, not causal proof.

The selector transfers when structure transfers



Holdout test

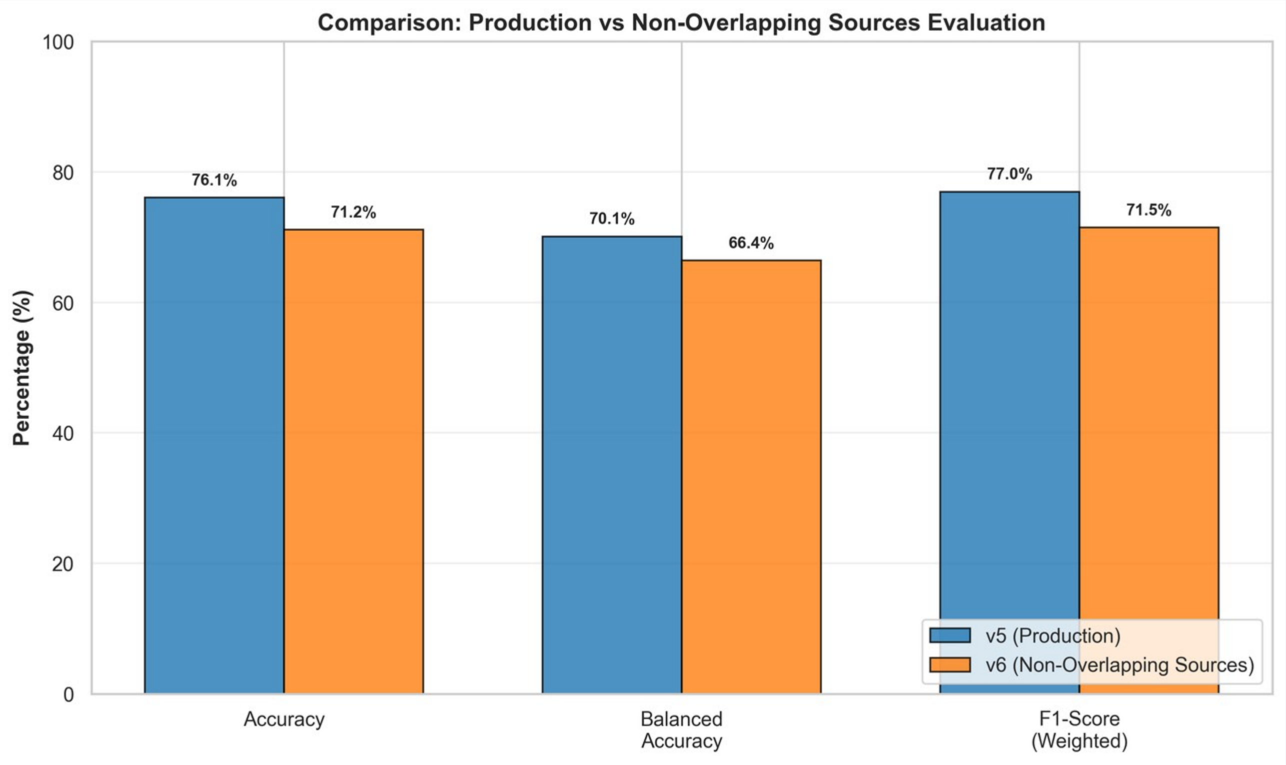
not uniform

but runtime value

survives in several

domain removals

Strict checks changed the interpretation, not the c



source-aware split lowered the numbers

v7 regret-aware training was rejected

v8 exposed the intro-heap boundary

The stricter checks make the final claim narrower, but more defensible.

The remaining errors show where the features sto

Clear boundary

timsort vs the rest

Hard boundary

introsort vs heapsort

F1 channel routing is a separate specialization: useful to mention, not the main thesis mo

This keeps the three-label v5 selector and the F1-specific five-label experiment separated.

The contribution is a selector and an evaluation d

1. Dataset and labeling protocol

real domains, structural transforms, measured winners

2. Structural ML selector

v5 XGBoost model using 16 cheap array features

3. Runtime-aware evaluation

accuracy, regret, SBS, VBS, and gap closed together

The next step is hardware-aware and adapti

hardware context

cache, CPU, memory behavior

adaptive selection

future online learning

broader portfolio

more algorithms, stricter splits

?

Thank you

I am ready for your questions.